

ПРОЕКТНАЯ РАБОТА

Тема проектной работы

«УДАЛЕНИЕ ШУМА С ИСПОЛЬЗОВАНИЕМ РЕГУЛЯРИЗАЦИИ МОРО-ЙОШИДЫ»

Студенты группы КМБО-04-23

Адамович И.О.

Друк Е.А.

Егоров Д.В.

Кочуркова А.В.

Стебалов Д.С.

Торосян Д.А.

Акаев А.Ш.

Студентка группы КМБО-03-23

Черткова М.С.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
ГЛАВА 1. Поиск алгоритмов для реализации	4
ГЛАВА 2. Описание алгоритмов.....	5
ГЛАВА 3. Результаты применения и сравнение	12
ЗАКЛЮЧЕНИЕ	20
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	21
ПРИЛОЖЕНИЕ	22

ВВЕДЕНИЕ

Как нам известно, задачи, связанные с улучшением качества изображений, такие как удаление шума, восстановление деталей и улучшение контраста, имеют широкий спектр применения в медицине, робототехнике, компьютерном зрении и многих других отраслях. Одной из важных задач обработки изображений является удаление шума (denoising), которое направлено на минимизацию влияния нежелательных артефактов, сохраняя при этом важные детали изображения. Цель данной работы — исследование и реализация методов для удаления шума с использованием TV-нормы и регуляризации Моро-Йошиды.

ГЛАВА 1. Поиск алгоритмов для реализации

Шум в изображениях возникает по различным причинам: ограничения оборудования (например, чувствительность датчиков камеры), условия съемки (низкая освещённость) или передача данных.

Также, классические методы удаления шума, например фильтры Гаусса или медианные фильтры, часто неэффективны для сохранения мелких деталей и резких границ. Это особенно важно в задачах, где требуется высокая точность восстановления изображения.

Одним из популярных подходов к решению этой задачи является использование вариационных методов, основанных на минимизации функционалов, таких как TV-норма (Total Variation). Однако классические методы, основанные на TV-норме, могут приводить к проблемам, связанным с не сглаженностью целевой функции, что затрудняет численное решение. Для решения этой проблемы применяется регуляризация Моро-Йошиды, которая позволяет сгладить целевую функцию, сохраняя её ключевые свойства.

Кроме того, в нашей работе уделяется внимание сравнению эффективности этих методов по метрикам PSNR (Peak Signal-to-Noise Ratio) и SSIM (Structural Similarity Index).

ГЛАВА 2. Описание алгоритмов

1.1 Понятие регуляризации

Регуляризация — это метод, используемый для улучшения обобщающей способности модели. Он добавляет дополнительные ограничения или информацию в процесс обучения. В контексте обработки сигналов регуляризация помогает сохранить важные характеристики данных, минимизируя влияние шумов. Мы используем регуляризацию TV (Total Variation), так как она является одним из наиболее популярных методов регуляризации в задачах обработки изображений и восстановления сигналов. Этот метод направлен на сохранение резких границ и деталей в изображениях, одновременно уменьшая шум. Основная идея TV-регуляризации заключается в минимизации общего вариационного расстояния, что позволяет эффективно справляться с шумами, сохраняя при этом важные структуры данных.

Для задачи минимизации будет использоваться градиентный спуск.

Пусть задана функция:

$$F(\vec{x}) : X \rightarrow \mathbb{R}$$

И задача минимизации задана следующим образом:

$$F(\vec{x}) \rightarrow \min_{\vec{x} \in X}$$

Основная идея градиентного спуска в том, чтобы двигаться обратно от направления скорейшего роста функции — антиградиенту $-\nabla F$. Тогда градиентный спуск можно определить следующим образом:

$$\overrightarrow{x^{[J+1]}} = \overrightarrow{x^{[J]}} - \lambda^{[J]} \nabla F(\overrightarrow{x^{[J]}}) \quad (1)$$

где $\overrightarrow{x^{[j+1]}}$ - обновленное значение, $\overrightarrow{x^{[j]}}$ - предыдущее, $\lambda^{[j]}$ - параметр, регулирующий скорость обучения, $\nabla F(\overrightarrow{x^{[j]}})$ - градиент функции по предыдущему значению.

Для минимизации мы будем использовать модификацию градиентного спуска - градиентный спуск с импульсом.

Определяется следующими формулами:

$$v(t) = \alpha v(t-1) + \delta \nabla_{\theta} (J(\theta(t))) \quad (2)$$

$$\theta(t+1) = \theta(t) - v(t) \quad (3)$$

где $v(t)$ — градиент на данной итерации с учетом предыдущих градиентов, δ — скорость обучения, $\nabla_{\theta} (J(\theta(t)))$ - градиент, $\theta(t+1)$ - шаг градиентного спуска.

α — параметр для предыдущих значений градиента. Обычно близок к 0.1.

Данный метод учитывает вклад предыдущих шагов обучения, а не только на данной итерации. Как правило, данный метод используют для негладких функций с множеством локальных минимумов, что соответствует предмету рассмотрения нашей задачи.

Общая вариация функции u в многомерном пространстве определяется как:

$$TV(u) = \int_{\Omega} |\nabla u(x)| dx \quad (4)$$

Выведем формулу для приложения к изображениям:

Производная в рамках работы с изображениями определяется, как разность между соседними пикселями:

$$u'_x(x, y) = u(x+1, y) - u(x, y) \quad (5)$$

$$u'_y(x, y) = u(x, y+1) - u(x, y) \quad (6)$$

Данные формулы следуют из формального определения производной, где приращение функции стремится к единице.

Из определения следует, что определенный интеграл — сумма бесконечно малых дифференцируемых промежутков на интервале.

Тогда формула TV для одномерного случая с функцией с действительными значениями определяется следующей формулой:

$$TV(f(x)) = \sum_{i=1}^n |f(x_{i+1}) - f(x_i)|_p^q \quad (7)$$

Так как работа с изображением подразумевает двумерное пространство, сумма разностей будет вычисляться для двух осей:

$$TV(f(x)) = \sum_{i,j \in N} |f(x_i) - f(x_j)|_p^q \quad (8)$$

TV отвечает за сохранение контуров и границ в изображении.

В контексте задачи регуляризации мы будем использовать градиент по TV.

Он определяется как:

$$D = \begin{bmatrix} -1 & 1 & \dots & 0 \\ \vdots & & \ddots & \vdots \\ 0 & & \dots & 1 \end{bmatrix} \quad (9)$$

Где D - матрица разностей между соседними пикселями.

D соответствует матрице, составленной из разностей, определенных в общей формуле TV.

Для сохранения углов мы используем TV, а для меры схожести двух изображений — разность между найденным значением и имеющимся, вычисленную в L2 норме:

$$|u - y|_2^1 = (\sum_{i=1} (u - y)_i^2)^{\frac{1}{2}} \quad (10)$$

Что соответствует стандартной Евклидовой норме (длина вектора в Евклидовом пространстве)

В данной форме будет вычисляться метрика, для использования же градиента мы определим следующее выражение:

$$\frac{1}{2}(u - v) \quad (11)$$

Тогда мы можем вывести общую формулу регуляризации TV:

$$tvd(y; \lambda) = \min_{x \in R^N} \left\{ \frac{1}{2} * \|y - x\|_2^2 + \lambda * \|D * x\|_1 \right\} \quad (12)$$

Где мы минимизируем TV и разницу в L2 нормы, используя ранее выведенные градиенты. λ — параметр, регулирующий скорость обучения.

1.3 Метод Моро-Йошиды

$$M_{\mu}f(x) = \min_y \left(f(y) + \frac{1}{(2\mu)} (\|x - y\|)^2 \right) \quad (13)$$

оболочка Моро-Йошиды (см. формулу 13.) (см. источник 4).

μ — параметр, регулирующий шаг обучения; y — минимизируемое значение; x — исходное значение; $f(y)$ — целевая функция.

Метод Моро-Йошиды представляет собой подход к регуляризации, основанный на минимизации значения в “оболочке” (см. формулу 13). Он был разработан для решения задач минимизации выпуклых функций и находит широкое применение в различных областях, включая обработку изображений и восстановление сигналов. Основная идея в том, чтобы восстановить объект, минимизируя по градиентам хорошо дифференцируемую разницу в L2 норме между восстанавливаемым объектом и оригиналом и функцию, применяемую к объекту. В контексте удаления шума метод Моро-Йошиды позволяет эффективно восстанавливать сигнал, минимизируя влияние шумов и сохраняя важные характеристики данных.

Метод Моро-Йошиды можно описать следующими ключевыми аспектами:

Выпуклые множества: обладают таким свойством, что для любых двух точек в множестве вся линия, соединяющая эти точки, также принадлежит этому множеству.

Проекция: проекция на множество допустимых решений позволяет находить оптимальные решения, которые соответствуют заданным ограничениям. Проекция может быть выполнена с использованием различных алгоритмов, таких как метод градиентного спуска.

Функция потерь: в методе Моро-Йошиды функция потерь включает как отклонение восстановленного сигнала от исходного, так и регуляризационный член, отвечающий за сохранение структуры данных. Это позволяет находить баланс между точностью восстановления и устойчивостью к шуму.

Если вычисление разницы и регуляризационного члена происходит в L2 норме, то нет принципиальной разницы между регуляризацией Моро-Йошиды и Total Variation, потому что в данной норме функция является гладкой. В случае L1 нормы Total Variation обычно не рассматривается, а используется оболочка Моро-Йошиды в применении к одному из членов минимизации (см. источник 6). В дальнейшем рассмотрении мы будем применять оболочку к Total Variation, вычисленной в L1 норме.

Согласно источнику оболочку Моро-Йошиды в рамках применения к Total Variation, можно рассмотреть, как градиентный спуск по Total Variation, где градиент — это повторное вычисление матрицы разниц (см. формулу 8.).

$$s_{\alpha}(x) = \min_y \left((\|D\|_1)^2 + \frac{\alpha}{2} \|x - v\|_2^2 \right) \quad (14)$$

оболочка Моро-Йошиды в рамках задачи (см. формулу 13.).

Тогда регуляризация с использованием Моро-Йошиды может быть выражена так (см. формулу 14) (см. источник 5):

$$s_{\alpha}(x) = \min_y \left(\|Dtv_d \left(1, \frac{1}{\alpha} \right) \|_1 + \frac{\alpha}{2} \|x - v\|_2^2 \right) \quad (15)$$

1.4 Метрики

MSE (Mean Squared Error) – среднеквадратичная ошибка, является популярной метрикой в статистике и машинном обучении.

$$MSE = \frac{1}{m*n} \sum_{i=1}^m \sum_{j=1}^n (X_{ij} - Y_{ij})^2 \quad (16)$$

SSIM (Structural Similarity Index Measure) — это метрика, предназначенная для оценки структурного сходства между двумя изображениями. В отличие от традиционных метрик, таких как MSE или PSNR, SSIM стремится имитировать восприятие изображений человеческим глазом, фокусируясь на яркости, контрасте и структуре изображения. Чем ближе её значение к единице, тем лучше полученный результат обработки изображения.

Метрика SSIM рассчитывается по формуле:

$$SSIM(x, y) = \frac{(2*\mu_x*\mu_y+c_1)(2*\sigma_{x*y}+c_2)}{(\mu_x^2+\mu_y^2+c_1)(\sigma_x^2+\sigma_y^2+c_2)} \quad (17)$$

Где:

- μ_x, μ_y — средние значения яркости изображений;
- σ_x^2, σ_y^2 — дисперсии изображений;
- σ_{xy} — ковариация между изображениями;
- C_1 и C_2 — малые постоянные, предотвращающие деление на ноль.

Диапазон значений SSIM:

- 1.0 — изображения идентичны;
- 0.0 — отсутствует структурное сходство;
- < 0 — изображения структурно различны.

PSNR (Peak Signal-to-Noise Ratio) — это метрика, широко используемая для оценки качества сжатых или изменённых изображений по сравнению с оригиналом. Она выражает отношение максимальной мощности сигнала (оригинального изображения) к мощности шума (разницы между оригиналом и сжатым изображением).

$$PSNR = 10 * \left(\frac{MAX_I^2}{MSE} \right) \quad (18)$$

Где MAX_I^2 - максимальное значение пикселя (для 8-битных изображений обычно 255). Чем выше значение PSNR, тем ближе обработанное изображение к оригиналу. Типичные значения PSNR для качественных изображений составляют интервал от 30 до 50 дБ. Если PSNR очень высок (например, 100 дБ), значит изображения практически идентичны.

ГЛАВА 3. Результаты применения и сравнение

Сравнение шло между моделью L2-TV и Moreau-Yoshida над L1-TV.

Тестирование проводилось на 3 изображениях из стандартного набора (тут перечисление). Были протестированы следующие параметры: *strength* (общий параметр для основного цикла обучения) в диапазоне $[0.1, 0.9]$ с шагом 0.1 и *mu* $[0.001, 0.01]$ (параметр для оболочки Моро-Йошиды над Total Variation).

Данные параметры могут принимать любое значение, кроме нуля. Однако рекомендуется выбирать значения, не превышающие максимальные в диапазоне тестирования, чтобы не получить слишком большой шаг градиентного спуска. Увеличить диапазон за счет уменьшения нижней границы принимаемых значений позволительно в случае увеличения количества итераций обучения. Для Гауссова шума была выбрана дисперсия 0.0025 ($\sigma=0.05$), для шума соль-перец будет подписана дисперсия индивидуально для каждого изображения ввиду случайности распределения. Параметр *amount* = 0.05 отвечает за процент измененных пикселей, а не за дисперсию распределения шума.

Удобная схема работы алгоритма доступна тут: [схема](#)

Для изображения *airplane.bmp*:

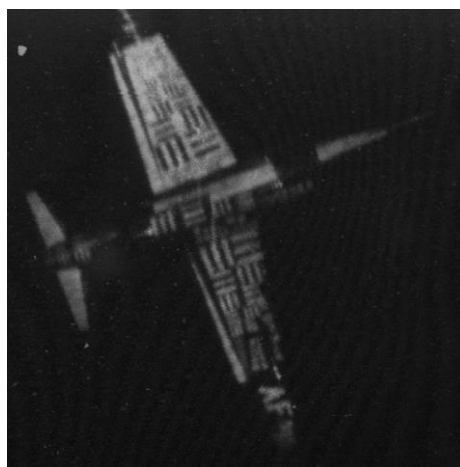
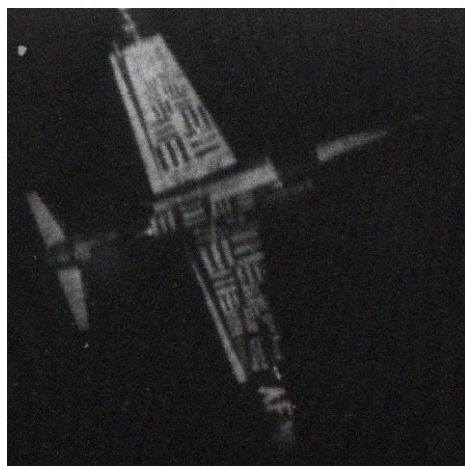


Рис. 1.1: изображение *airplane.bmp*

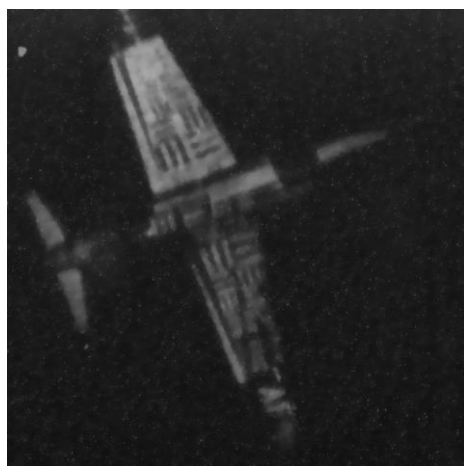


a



b

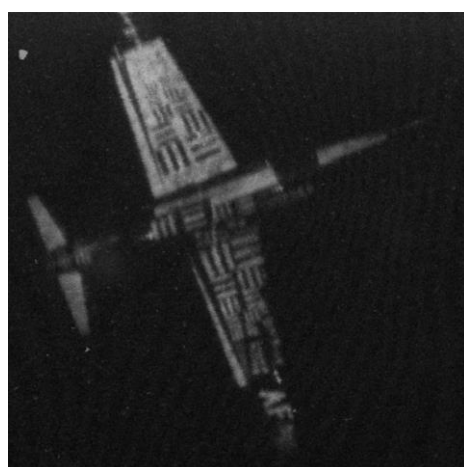
Рис. 1.2: a) изображение *airplane.bmp* с шумом типа соль-перец, дисперсия: 5.66; b) изображение *airplane.bmp* с Гауссовым шумом



a



b



c



d

Рис. 1.3:

- a) Лучший результат обработки шума типа соль-перец с помощью Моро-Йошиды (параметры: $strength=0.1$, $mu=0.001$; метрики: $ssim:0.073$, $psnr:31.698$)
- b) Лучший результат обработки шума типа соль-перец с помощью L2-TV-регуляризации (параметры: $strength=0.2$, $mu=0.002$; метрики: $ssim: 0.052$, $psnr:31.407$)
- c) Лучший результат обработки Гауссова шума с помощью Моро-Йошиды (параметры: $strength=0.1$, $mu=0.001$; метрики: $ssim: 0.384$, $psnr:34.021$)
- d) Лучший результат обработки Гауссова шума с помощью L2-TV-регуляризации (параметры: $strength=0.1$, $mu=0.001$; метрики: $ssim: 0.077$, $psnr: 34.430$)

Вывод: для данного типа шума Моро-Йошида лучше по обоим типам шума. С одной стороны, небольшую текстуру шума он все еще сохраняет, с другой - лучше восстанавливает детали без лишнего размытия.

Для изображения *fruits.bmp*:



Рис. 2.1: изображение *airplane.bmp*



a

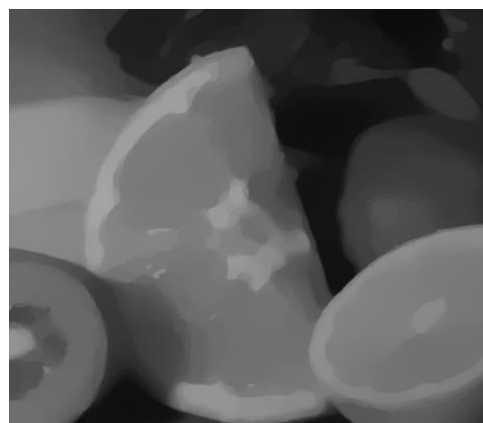


b

Рис. 2.2: a) изображение *fruits.bmp* с шумом типа соль-перец, дисперсия: 4.93; b) изображение *fruits.bmp* с Гауссовым шумом



a



b



c



d

Рис. 2.3:

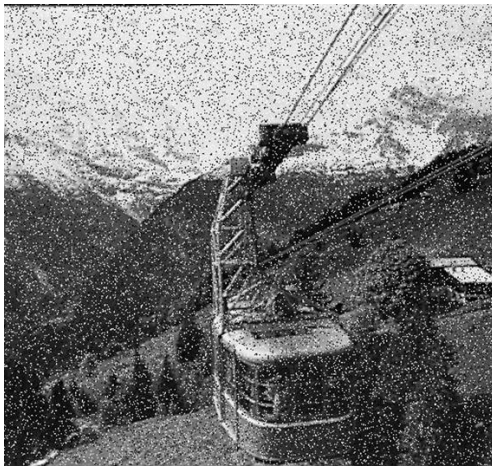
- a) Лучший результат обработки шума типа соль-перец с помощью Моро-Йошиды (параметры: $strength=0.6$, $\mu=0.006$; $ssim:0.255$, $psnr: 31.149$)
- b) Лучший результат обработки шума типа соль-перец с помощью L2-TV-регуляризации (параметры: $strength=0.1$, $\mu=0.001$; метрики $ssim: 0.231$ $psnr:30.944$)
- c) Лучший результат обработки Гауссова шума с помощью Моро-Йошиды (параметры: $strength=0.2$, $\mu=0.002$; метрики: $ssim: 0.338$, $psnr:32.240$)
- d) Лучший результат обработки Гауссова шума с помощью L2-TV-регуляризации (параметры: $strength=0.1$, $\mu=0.001$; метрики: $ssim: 0.316$, $psnr: 31.884$)

Вывод: в обоих случаях лучше справился метод Моро-Йошиды.

Для изображения *cablecar.bmp*:



Рис. 3.1: изображение *cablecar.bmp*



a



b

Рис. 3.2: a) изображение *cablecar.bmp* с шумом типа соль-перец, дисперсия: 5.46; b) изображение *cablecar.bmp* с Гауссовым шумом



a



b



c



d

Рис. 3.3:

- a) Лучший результат обработки шума типа соль-перец с помощью Моро-Йошиды (параметры: $strength=0.2$, $\mu=0.002$; $ssim: 0.3954$, $psnr: 28.59$)
- b) Лучший результат обработки шума типа соль-перец с помощью L2-TV-регуляризации (параметры: $strength=0.1$, $\mu=0.001$; метрики $ssim: 0.242$ $psnr: 30.126$)
- c) Лучший результат обработки Гауссова шума с помощью Моро-Йошиды (параметры: $strength=0.5$, $\mu=0.005$; метрики: $ssim: 0.595$, $psnr: 32.700$)
- d) Лучший результат обработки Гауссова шума с помощью L2-TV-регуляризации (параметры: $strength=0.1$, $\mu=0.001$; метрики: $ssim: 0.293$, $psnr: 30.982$)

Вывод: в данном случае с Гауссовым шумом лучше справляется Моро-Йошида, а с шумом типа соль-перец - L2-TV-регуляризация.

На примере рассмотренных изображений можно заключить, что Моро-Йошида лучше справляется как с шумом типа соль-перец, так и с более плотным шумом - Гауссовым шумом по той причине, что в оболочку у нас заключена L1-TV-функция, считающая разность соседних пикселей. А L2-TV-регуляризация показала себя лучше только в 1 из 3 случаев (изображение cablecar.bmp) в шуме типа соль-перец.

Таблица 1.1. (сравнение SSIM-метрик для шума типа соль-перец)

Метод обработки	airplane.bmp	fruits.bmp	cablecar.bmp
Моро-Йошида	<u>ssim = 0.073</u> параметры: strength=0.1, mu=0.001	<u>ssim:0.255</u> параметры: strength=0.6, mu=0.006	<u>ssim:0.3954</u> параметры: strength=0.2, mu=0.002
L2-TV	<u>ssim = 0.052</u> параметры: strength=0.2, mu=0.002	<u>ssim: 0.231</u> параметры: strength=0.1, mu=0.001	<u>ssim: 0.242</u> параметры: strength=0.1, mu=0.001

Таблица 1.2. (сравнение SSIM-метрик для Гауссова шума)

Метод обработки	airplane.bmp	fruits.bmp	cablecar.bmp
Моро-Йошида	<u>ssim: 0.384</u> параметры: strength=0.1, mu=0.001	<u>ssim: 0.338</u> параметры: strength=0.2, mu=0.002	<u>ssim:0.595</u> параметры: strength=0.5, mu=0.005
L2-TV	<u>ssim: 0.077</u> параметры: strength=0.1, mu=0.001	<u>ssim:0.316</u> параметры: strength=0.1, mu=0.001	<u>ssim:0.293</u> параметры: strength=0.1, mu=0.001

ЗАКЛЮЧЕНИЕ

В рамках данной работы были исследованы и реализованы современные методы обработки изображений, основанные на TV-норме и регуляризации Моро-Йошиды. Эти методы эффективны для решения задач удаления шума, сохраняя при этом важные детали и резкие границы в изображениях.

Результаты исследования подтвердили эффективность применения регуляризации Моро-Йошиды для сглаживания негладких функций, таких как TV-норма. Это позволило преодолеть ограничения классических методов оптимизации и улучшить численную устойчивость алгоритмов. Использование проксимальных операторов и градиентного спуска с адаптивным шагом и импульсом обеспечило высокую скорость сходимости и качество восстановления изображений.

Оценка результатов по метрикам PSNR и SSIM продемонстрировала, что предложенный подход позволяет достичь значительного улучшения качества изображений даже в условиях сильного зашумления. В частности, регуляризация Моро-Йошиды показала свою эффективность в сравнении с классическими методами, основанными на TV-норме.

Практическая значимость работы заключается в возможности применения разработанных методов в широком спектре задач обработки изображений, таких как анализ спутниковых данных, улучшение качества фотографий и предобработка данных для систем машинного обучения.

Теоретическая ценность работы состоит в изучении свойств регуляризации Моро-Йошиды и её роли в задачах вариационной оптимизации. Полученные результаты могут быть использованы для дальнейшего развития методов обработки данных, основанных на выпуклом анализе и проксимальных операторах.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Lucet, Y. Fast Moreau envelope computation I: Numerical algorithms / Y. Lucet // Journal of Computational and Applied Mathematics. — 2007. — Vol. 200, Issue 1. — P. 235–246.
2. Sun, J. On a conjecture in Moreau-Yosida approximation of a nonsmooth convex function / J. Sun, D. Han // Journal of Optimization Theory and Applications. — 1997. — Vol. 94, Issue 3. — P. 741–747.
3. Meng, F. On second order properties of the Moreau-Yosida regularization for constrained nonsmooth convex programs / F. Meng, G. Zhao // Mathematical Programming. — 2004. — Vol. 101, Issue 2. — P. 377–400.
4. Zheng, X. Truncated low-rank and total p-variation constrained color image completion and its Moreau approximation algorithm / X. Zheng et al. // IEEE Transactions on Image Processing. — 2019. — Vol. 28, Issue 8. — P. 3856–3868.
5. Selesnick, I. W. Total variation denoising via the Moreau envelope / I. W. Selesnick // IEEE Signal Processing Letters. — 2017. — Vol. 24, Issue 2. — P. 141–145.
6. Chen, C. The Moreau envelope approach for the L1/TV image denoising model / C. Chen, H. Shen, F. Xu, X. Zeng // Inverse Problems and Imaging. — 2014. — Vol. 8, Issue 1. — P. 1–27.
7. Lucic, V. Volatility interpolation and Yosida approximation / V. Lucic // Finance and Stochastics. — 2012. — Vol. 16, Issue 3. — P. 495–515.
8. Alves, M. M. Moreau-Yosida regularization of maximal monotone operators of type (D) / M. M. Alves, B. F. Svaiter // Set-Valued and Variational Analysis. — 2010. — Vol. 18, Issue 3–4. — P. 297–315.

ПРИЛОЖЕНИЕ

```
import numpy as np

import cv2

from skimage.io import imread

from skimage.color import rgb2gray

from skimage.metrics import structural_similarity as ssim


def validate_images(original, compressed):

    if original is None or compressed is None:

        raise ValueError("Одно или оба изображения не загружены")

    if original.size == 0 or compressed.size == 0:

        raise ValueError("Изображения не должны быть пустыми")

    if original.dtype != np.uint8 or compressed.dtype != np.uint8:

        raise TypeError("Оба изображения должны быть 8-битными")

    if original.shape != compressed.shape:

        raise ValueError("Изображения должны быть одинакового размера")


def to_grayscale(image):

    if image.ndim == 3:

        return rgb2gray(image)

    return image


def PSNR(original, compressed):

    validate_images(original, compressed)

    mse = np.mean((original - compressed) ** 2)

    if mse == 0:

        return float('inf')

    max_pixel = 255.0
```

```
return 20 * np.log10(max_pixel / np.sqrt(mse))
```

```
def SSIM(original, compressed):
```

```
    validate_images(original, compressed)
```

```
    gray1 = to_grayscale(original)
```

```
    gray2 = to_grayscale(compressed)
```

```
    score, _ = ssim(gray1, gray2, full=True, data_range=1.0)
```

```
    return score
```

```
def tv_norm(image, eps=1e-8, l=1, p=2):
```

```
    x_diff = image[:-1, :-1] - image[:-1, 1:]
```

```
    y_diff = image[:-1, :-1] - image[1:, :-1]
```

```
    grad_mag = (abs(x_diff**p) + abs(y_diff**p) + eps)**(1/p)
```

```
    loss = np.sum(grad_mag)
```

```
    dx_diff = x_diff / grad_mag
```

```
    dy_diff = y_diff / grad_mag
```

```
    grad = np.zeros_like(image, dtype=np.float64)
```

```
    grad[:-1, :-1] += dx_diff + dy_diff
```

```
    grad[:-1, 1:] -= dx_diff
```

```
    grad[1:, :-1] -= dy_diff
```

```
    return loss, grad
```

```
def lp_norm(image, orig_image, l, p):
```

```
    grad = (1/p)*(image - orig_image)
```

```
    loss = (np.sum(grad ** p)) ** (1/p)
```

```
    return loss, grad
```

```
def prox_tv(u, weight, iter=10):
```

```

v = tv_norm(u.copy(),p=1)[1]

for _ in range(iter):

    loss, grad = tv_norm(v, p=1)

    v = v - weight * grad

return v

def my_tv_envelope(u, mu=0.001, prox_iter=20):

    v = np.array(prox_tv(u, weight=mu, iter=prox_iter))

    loss, _ = tv_norm(v)

    envelope = loss + (1 / (2 * mu)) * np.sum((v - u) ** 2)

    grad = v

    return envelope, grad

def eval_loss_and_grad(image, orig_image, strength=0.9, mu=0.01, useMorea=False):

    lp_loss, lp_grad = lp_norm(image, orig_image, 1, 2)

    if useMorea: # if true => MoreaYosida

        tv_loss, tv_grad = my_tv_envelope(image, mu)

        loss = tv_loss + lp_loss

        grad = tv_grad + lp_grad

    else: # if false => TV

        tv_loss, tv_grad = tv_norm(image, l=1, p=2)

        loss = strength * tv_loss + lp_loss

        grad = strength * tv_grad + lp_grad

    return loss, grad

def tv_denoise_gradient_descent(image, strength=0.1, step_size=1e-2, tol=3.2e-3, iter=0,

                                mu=0.01, use=False):

```



```

orig_image = image.copy()

momentum = np.zeros_like(image)

momentum_beta = 0.9

loss_smoothed = 0

loss_smoothing_beta = 0.9

i = 0

if iter == 0:

    while True:

        i += 1

        loss, grad = eval_loss_and_grad(image, orig_image, strength, mu, use)

        loss_smoothed = loss_smoothed * loss_smoothing_beta + loss * (1 - loss_smoothing_beta)

        loss_smoothed_debiased = loss_smoothed / (1 - loss_smoothing_beta ** i)

        if i > 1 and loss_smoothed_debiased / loss < tol + 1:

            break

        step = step_size / (strength + 1)

        # momentum импульс

        # momentum_beta коэффициент к импульсу

        momentum = momentum * momentum_beta + grad * (1 - momentum_beta)

        image -= step / (1 - momentum_beta ** i) * momentum

    else:

        for i in range(1, iter):

            loss, grad = eval_loss_and_grad(image, orig_image, strength, mu, use)

            step = step_size / (strength + 1)

            momentum = momentum * momentum_beta + grad * (1 - momentum_beta)

            image -= step / (1 - momentum_beta ** i) * momentum

```

```

return image

# Основной запуск

if __name__ == "__main__":

    image_path = "shum.jpg"

    output_path = "denoised_my_3.jpg"

    output_path_tv = "denoised_my_3_tv.jpg"


    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE).astype(np.float32) / 255.0


    denoised = tv_denoise_gradient_descent(image.copy(), strength=0.1, iter=20, mu=0.01, use=True)

    denoised_tv = tv_denoise_gradient_descent(image.copy(), strength=0.1, iter=100, mu=0.01, use=False)


    cv2.imwrite(output_path, (denoised * 255).clip(0, 255).astype(np.uint8))

    cv2.imwrite(output_path_tv, (denoised_tv * 255).clip(0, 255).astype(np.uint8))


    original = (image * 255).astype(np.uint8)

    denoised_uint8 = (denoised * 255).astype(np.uint8)

    denoised_uint8_tv = (denoised_tv * 255).astype(np.uint8)

    print(f"Moreau-Yosida → PSNR: {PSNR(original, denoised_uint8):.2f}, SSIM: {SSIM(original, denoised_uint8):.4f}")

    print(f"TV → PSNR: {PSNR(original, denoised_uint8_tv):.2f}, SSIM: {SSIM(original, denoised_uint8_tv):.4f}")

```